# Intuition for Diffusion in Nature and in Generative Modeling

## 1 Physical Diffusion and the Ubiquity of Bell-Shaped Distributions

Many everyday phenomena can be understood as the result of *lots of tiny random influences piling up*.

- **Dye spreading in water.** Each molecule gets nudged by countless collisions with water molecules. After many nudges, the cloud of dye becomes a smooth blur that expands over time.
- **Heat spreading in a metal rod.** Energy moves around via many microscopic interactions, producing a smooth "heat profile" that spreads out.
- **Measurement noise.** When a sensor is affected by many small independent sources of error, the total error often forms a bell-shaped curve.
- **Human height (roughly).** Height reflects many small contributions (genes, nutrition, health). When many small effects add up, the population often looks bell-shaped around a typical value.

A useful mental model is: *each outcome is the sum of many small pushes*, and the *sum tends to form a bell-shaped profile.*

In what follows, we will call this bell-shaped family the **normal curve** (in 1D) and the **normal cloud** (in multiple dimensions).

## 2 Predicting probabilities with an integral (1D)

Suppose a quantity $x$ is described by a *normal curve* centered at $\mu$ with **spread** $\sigma$ ("sigma"). The spread $\sigma$ controls how wide the bell is: a larger $\sigma$ means outcomes are more scattered around the center. A useful rule of thumb is that the interval $[\mu - 2\sigma,\, \mu + 2\sigma]$ captures approximately 95% of all outcomes. In $d$ dimensions the analogous statement is that a sphere of radius

$$r_{95} \;\approx\; \sigma + \sqrt{d}$$

centered at $\mu$ captures approximately 95% of the cloud. The extra $\sqrt{d}$ term reflects the fact that in high dimensions the cloud concentrates on a shell rather than near the center.

### Probability of landing in an interval

If you want the chance that $x$ lands between $a$ and $b$, you compute the *area under the curve* between $a$ and $b$:

$$\text{Probability}(a \le x \le b) \;=\; \int_a^b (\text{height of the normal curve at } x)\, dx.$$

Intuition:

- The curve is taller where outcomes are more likely.
- The integral adds up "tiny rectangles" of probability from $a$ to $b$.

In practice, people often use a lookup table or software for this integral, but conceptually it is just *area under the bell*.

## 3 Predicting probabilities with an integral (multiple dimensions)

Now let $x$ be a *vector* (e.g., a 2D location or a high-dimensional image vector). The normal curve becomes a *normal cloud*: it assigns a *density* to each point in space.

### Probability of landing in a region

If $R$ is a region (like a rectangle, a circle, or any shape), then

$$\text{Probability}(x \in R) \;=\; \int_{x \in R} (\text{density of the normal cloud at } x)\, dx.$$

Intuition:

- Replace "area under a curve" with "volume under a surface" (or, in higher dimensions, "mass in a region").
- You add up tiny pieces of probability mass across the region.

### A common special case: inside a ball

If the cloud is symmetric around the origin (equally spread in every direction), then a very simple question is:

$$\text{Probability}(\|x\| \le r) \;=\; \int_{\|x\| \le r} \text{density}(x)\, dx.$$

This is "how much probability mass is inside a radius-$r$ ball."

## 4 How to simulate samples using uniforms + adding up (intuition)

To *simulate* outcomes from these bell-shaped dynamics, we need a way to generate the required random noise. Assume we only have access to a **uniform random number generator** that returns values in $[-1, 1]$.

### The add-up trick (simple and intuitive)

A classic intuition is:

> *If you add many independent small random numbers, the result starts looking bell-shaped.*

Each draw $u_i \sim \text{Uniform}[-1, 1]$ has mean 0. Mathematics tells us that the sum $S = u_1 + \cdots + u_n$ increasingly resembles a standard bell-shaped noise as $n$ grows, and the rescaled value

$$z = S \cdot \sqrt{\tfrac{3}{n}}$$

behaves like a standard bell-shaped noise. A convenient choice is $n = 12$, giving $z = S/2$.
To get a bell-shaped value centered at $\mu$ with spread $\sigma$, use:

$$x = \mu + \sigma z.$$

### Multidimensional sampling: isotropic cloud

The simplest multivariate case is an **isotropic** (equally spread in every direction) normal cloud. Since there are no correlations between coordinates, the recipe is just:

1. Generate $d$ independent bell-shaped noises $z_1, \ldots, z_d$ (using the add-up trick above).
2. Assemble them into a vector $z = (z_1, \ldots, z_d)$.
3. Scale and shift: $x = \mu + \sigma z$.

Each coordinate gets its own independent nudges; the result is a spherical cloud centered at $\mu$.

## 5 Now: simulating a diffusion process (forward corrupts, reverse restores)

A **diffusion process** (in the generative-modeling sense) is a controlled way to:

- **Forward:** gradually corrupt structured data into pure noise using many small steps.
- **Reverse:** learn to undo those steps, removing noise little-by-little to recover structure.

We will keep things intuitive by writing only **sampling equations** (recipes for generating the next state).

### Ingredients

We choose a schedule of small noise amounts:

$$\beta_1, \beta_2, \ldots, \beta_T \quad \text{(each is small, between 0 and 1)}$$

and define

$$\alpha_t = 1 - \beta_t.$$

We also use a source of **standard noise**:

- In 1D: a scalar noise value $\varepsilon$
- In $d$ dimensions: a noise vector $\varepsilon$ whose coordinates are independent bell-shaped noises

You can simulate $\varepsilon$ using the **uniform-add-up trick** from the previous section.

## 6 Forward process: one-step and multi-step (sampling-only)

### One-step forward (add a small bit of noise)

Given $x_{t-1}$, generate the next noisy version $x_t$ by:

$$x_t \;=\; \sqrt{\alpha_t}\, x_{t-1} \;+\; \sqrt{1-\alpha_t}\, \varepsilon_t,$$

where $\varepsilon_t$ is freshly sampled standard noise.
Intuition:

- The factor $\sqrt{\alpha_t}$ slightly fades the old signal.
- The new term injects fresh randomness.

### Multi-step forward (direct jump to step $t$)

If you do many small corruptions, you can write $x_t$ directly from the original $x_0$:

$$x_t \;=\; \sqrt{\bar{\alpha}_t}\, x_0 \;+\; \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \bar{\alpha}_t \;=\; \alpha_1 \alpha_2 \cdots \alpha_t,$$

where $\varepsilon$ is standard noise.
This says:

- a shrinking fraction of the original survives,
- the rest becomes noise.

**Proof (for the curious reader)**

*(This derivation can be skipped on first reading.)*

**Claim.** Applying the one-step rule $t$ times yields

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

**Derivation by induction.**
*Base case ($t = 1$).* By the one-step rule,

$$x_1 = \sqrt{\alpha_1}\, x_0 + \sqrt{1-\alpha_1}\, \varepsilon_1.$$

This matches the claim with $\bar{\alpha}_1 = \alpha_1$.

*Inductive step.* Assume the claim holds at step $t-1$:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \varepsilon', \qquad \varepsilon' \sim \mathcal{N}(0, I).$$

Apply the one-step rule to go from $x_{t-1}$ to $x_t$:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_t = \sqrt{\alpha_t}\Big(\sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \varepsilon'\Big) + \sqrt{1-\alpha_t}\, \varepsilon_t.$$

Collecting the $x_0$ term and the noise terms:

$$x_t = \sqrt{\alpha_t \bar{\alpha}_{t-1}}\, x_0 + \underbrace{\sqrt{\alpha_t(1-\bar{\alpha}_{t-1})}\, \varepsilon' + \sqrt{1-\alpha_t}\, \varepsilon_t}_{\text{sum of two independent bell-shaped noises}} .$$

The sum of two independent bell-shaped noises with variances $a^2$ and $b^2$ is itself a bell-shaped noise with variance $a^2 + b^2$. Here $a^2 = \alpha_t(1-\bar{\alpha}_{t-1})$ and $b^2 = 1-\alpha_t$, so the combined noise has variance

$$\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t) = \alpha_t - \alpha_t\bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \alpha_t\bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t.$$

Therefore

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon,$$

which completes the induction. □

# 7 Reverse process (model): one-step and multi-step (sampling-only)

### Step 1: the ideal reverse formula (if we knew $\varepsilon$)

Recall the multi-step forward formula:

$$x_t \;=\; \sqrt{\bar{\alpha}_t}\, x_0 \;+\; \sqrt{1-\bar{\alpha}_t}\, \varepsilon.$$

If we happened to know the noise $\varepsilon$ that was used to produce $x_t$, we could solve for $x_0$ exactly:

$$x_0 \;=\; \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon}{\sqrt{\bar{\alpha}_t}}.$$

Substituting this into the one-step forward rule (run backwards) gives the *ideal* reverse step:

$$x_{t-1} \;=\; \frac{1}{\sqrt{\alpha_t}} \left( x_t \;-\; \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon \right) \;+\; \sqrt{\beta_t}\, z,$$

where $z$ is a small fresh noise injection that keeps the step statistically consistent. This formula is exact: given $\varepsilon$, it recovers $x_{t-1}$ perfectly.

### Step 2: the problem — $\varepsilon$ is unknown at inference time

During generation we start from pure noise and have no access to any real $x_0$, so the noise $\varepsilon$ that "would have" produced our $x_t$ is unknown. We must *estimate* it.

### Step 3: learned denoising — replace $\varepsilon$ by $\widehat{\varepsilon}_\theta$

We train a neural network $\widehat{\varepsilon}_\theta(x_t, t)$ to predict $\varepsilon$ given only $x_t$ and the time step $t$. Substituting the network's prediction for the unknown $\varepsilon$ gives the formula we actually use:

$$x_{t-1} \;=\; \frac{1}{\sqrt{\alpha_t}} \left( x_t \;-\; \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \cdot \widehat{\varepsilon}_\theta(x_t, t) \right) \;+\; \sqrt{\beta_t}\, z,$$

where:

- $\widehat{\varepsilon}_\theta(x_t, t)$ is the network's estimate of the noise $\varepsilon$ inside $x_t$,
- $\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$ is a known scalar from the schedule,
- $z$ is fresh standard noise (set to 0 on the final step),
- $\sqrt{\beta_t}$ controls the small randomness injected at each reverse step.

**Proof (for the curious reader): derivation of the reverse step**

*(This derivation can be skipped on first reading.)*

**Goal.** Show that the reverse step formula follows from the multi-step forward formula and the one-step forward rule.

**1. Recover $x_0$ from $x_t$ and $\varepsilon$.** The multi-step forward formula states

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon.$$

Solving for $x_0$:

$$x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon}{\sqrt{\bar{\alpha}_t}}. \tag{$*$}$$

**2. Write $x_{t-1}$ using the multi-step formula at $t-1$.** Applying the multi-step formula one step earlier:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \varepsilon'.$$

For the reverse step we want the *mean* of $x_{t-1}$ given $x_t$ (ignoring the residual noise $\varepsilon'$ for the moment), so we substitute $(*)$:

$$\mathbb{E}[x_{t-1} \mid x_t] = \sqrt{\bar{\alpha}_{t-1}} \cdot \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon}{\sqrt{\bar{\alpha}_t}}.$$

**3. Rewrite using $\alpha_t = \bar{\alpha}_t / \bar{\alpha}_{t-1}$.** Note that $\sqrt{\bar{\alpha}_{t-1}}/\sqrt{\bar{\alpha}_t} = 1/\sqrt{\alpha_t}$. Expanding:

$$\mathbb{E}[x_{t-1} \mid x_t] = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\sqrt{1-\bar{\alpha}_t}}{\sqrt{\alpha_t} \cdot \frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_{t-1}}}}\, \varepsilon \right) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon \right),$$

where the last equality uses $\beta_t = 1 - \alpha_t$ and the algebraic identity

$$\frac{1-\bar{\alpha}_t}{\sqrt{\alpha_t}} - \frac{\sqrt{(1-\bar{\alpha}_t)(1-\bar{\alpha}_{t-1})}}{\sqrt{\bar{\alpha}_t/\bar{\alpha}_{t-1}}} = \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}.$$

**4. Add back the residual randomness.** The true posterior of $x_{t-1}$ given $x_t$ and $x_0$ is a bell-shaped cloud centred at the mean above, with variance $\beta_t$ (the DDPM simplest choice). Adding the residual noise $\sqrt{\beta_t}\, z$, $z \sim \mathcal{N}(0, I)$, gives the full reverse sampling formula:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon \right) + \sqrt{\beta_t}\, z.$$

Replacing the unknown $\varepsilon$ by its network estimate $\widehat{\varepsilon}_\theta(x_t, t)$ gives the formula used in practice. $\square$

### Multi-step reverse (the full generation procedure)

1. Start from pure noise: sample $x_T$ as standard noise.
2. For $t = T, T-1, \ldots, 1$:

$$x_{t-1} \;=\; \frac{1}{\sqrt{\alpha_t}}\left(x_t \;-\; \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \cdot \widehat{\varepsilon}_\theta(x_t, t)\right) \;+\; \sqrt{\beta_t}\, z.$$

3. Output $x_0$.

## 8 Conclusion: how training works (the key idea)

We train the diffusion model by repeatedly:

- taking a real object $x_0$,
- corrupting it to some step $t$ using the forward sampling rule,
- and teaching a neural network to **predict the noise** that was used.

In other words:

> *The network learns to estimate "how much noise is inside this noisy object at step $t$,"* and that estimate is exactly what the reverse sampling step needs in order to denoise.